

LLM-Driven End-to-End PCB Generation for KiCad

PCBSchemaGen

Natural-language prompt → ERC-validated SKiDL netlist → native KiCad 9 schematic →
DRC-checked routed PCB

Dushyant Singh

President, Semiconductor Chip Design Club • SRM IST

Mentor: **Saurabh Rawat**, STMicroelectronics

Technical Review • September 3, 2026

Agenda

Part	Topic	Time
1	What the project does — flow, architecture, and the outputs generated (schematic + layout)	15 min
2	How to replicate the work — environment, dependencies, SKiDL, KiCad, LLM APIs	15 min
3	Next tasks — open gaps, roadmap, and the north-star benchmark	15 min
4	<i>Questions & discussion</i>	15 min

Total: 60 minutes. Backup material in the appendix for detailed questions.

Part 1

What the project does

The flow, the architecture, and the outputs generated

15 minutes

Today's hardware flow is human-serial

- Read datasheet → pick parts → draw schematic → place → route → check.
- Every stage is manual, and every stage is where a silent error enters.
- A decoupling cap on the wrong rail, a swapped SPI pin — none of it is caught until the board is fabricated.

Meanwhile, in digital design

- RTL → GDSII is automated end-to-end and *verified at every handoff*.
- PCB design has no equivalent closed loop.

The question this project answers

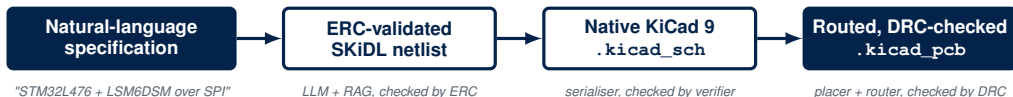
Can a language model be placed at the *front* of a hardware flow — as the **intent** stage — while deterministic, authoritative tools own every stage after it?

Design principle

The LLM proposes. **KiCad, SKiDL and the DRC engine dispose.**

No LLM output ever reaches a file without passing a machine check.

What PCBSchemaGen is



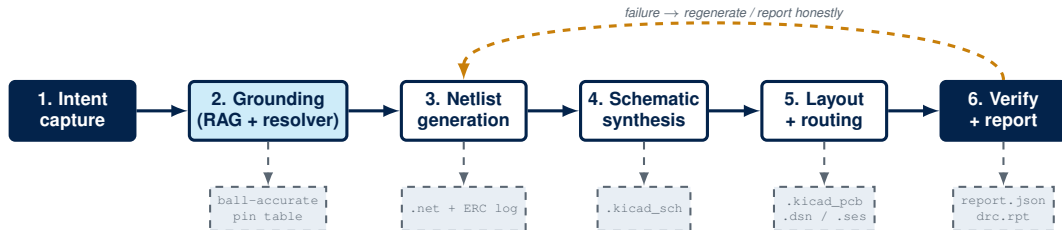
Scope

- Target vendor: **STMicroelectronics** parts
- Target tool: **KiCad 9** (native file formats, no round-trip through third-party formats)
- Fully headless — no GUI step anywhere in the flow

Benchmark circuit

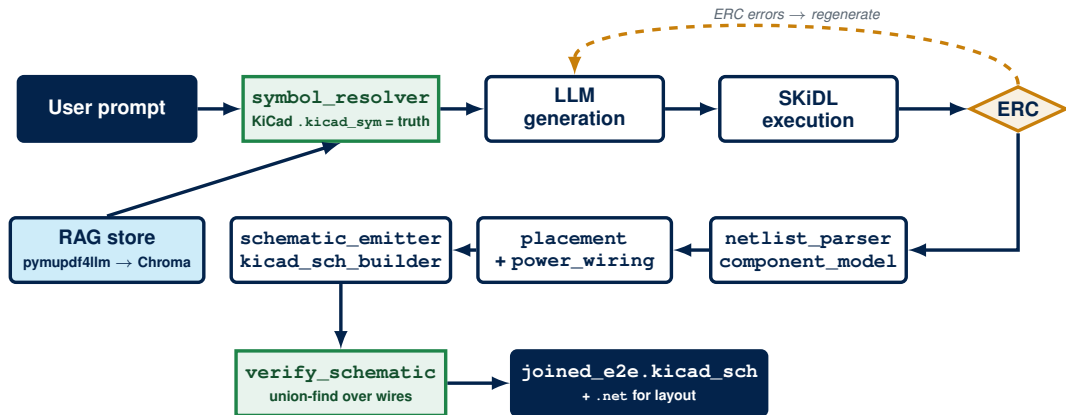
- **STM32L476JGYxP** (WLCSP-72) ↔ **LSM6DSM** IMU (LGA-14), SPI
- 22 components • 68 nets • 126 pads
- 7 independent supply rails

End-to-end flow — the six stages



Every arrow crosses a **verification gate**. A stage may only hand forward what the next stage's authoritative tool has accepted.

Phase 1 — schematic generation (detailed flow)



Two correctness problems, kept strictly separate: *netlist correctness* is the LLM's job (ERC judges it); *schematic-file correctness* is the serialiser's job (the verifier judges it).

Grounding: why the LLM never invents a pin

The failure we designed out

A model asked for "STM32L476 SPI to an IMU" will happily emit `U1[ 'PA5' ]` — a name that does not exist on a WLCSP-72 package, where pins are *ball designators* (A1, H8, G7...).

The fix: authoritative pin injection

- `kicad_symbol_parser.py` reads KiCad's own `.kicad_sym` files.
- `symbol_resolver.py` produces the exact, ball-accurate pin list for the requested part.
- That list is injected into the prompt, with a hard instruction: *use only these strings; never invent, never index numerically.*

Authoritative sources own truth

KiCad	owns symbols & footprints
ST	owns design intent
Resolver	owns pin data
Fab	owns DFM floors

RAG v2

`pymupdf4llm` for layout-aware markdown extraction.

Plain PyMuPDF mangled the ball-assignment tables into mojibake — exactly the data the model most needs.

Derived by analysing a real ST reference schematic (**STEVAL-PLC1MIC1**) and encoding its drawing convention.

The atomic ST unit

supply symbol → wire → junction dot → IC pin,
decoupling cap tapped at the junction,
GND symbol directly beneath.

Exact geometry (all on the 1.27 mm grid)

pin → junction	6.35 mm
pin → supply symbol	12.70 mm
cap offset	7.62 mm

The routability rule

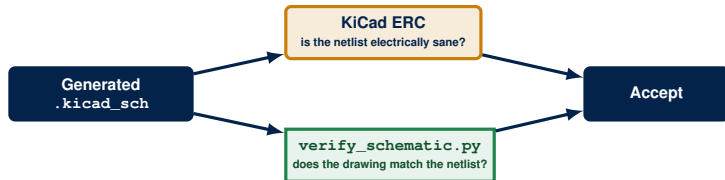
Wire a rail **if** its stacks fit the free space between the neighbouring rails' pins **and** cross no other pin.

Otherwise: label it, and place its caps in a dedicated supply region.

Result on the benchmark (7 rails)

- **5 wired A-rails:** VREF_PLUS, VBAT, VDDA, VDDIO2, VDDUSB
- **2 labelled B-rails:** VDD_3V3 (3 pins, 8 caps), VDD12 (2 pins)
- ✓ 0 ERC errors, 0 warnings

Verification: two independent judges



Two silent faults the verifier caught

1. **NRST and BOOT0 shorted to VREF_PLUS** — an edge-routing bug routed a LEFT-edge rail *along* the pin column. ERC saw nothing.
2. **GPIO PC8 touching a B-rail** — caught only because an *expected* floating-pin error went *missing* from the ERC report.

The principle worth remembering

A disappearing ERC error is worse news than a new one.

A new error means something broke loudly. A vanished one means two nets silently merged — and the check that would have complained is now satisfied by the short itself.

Output 1 — the generated schematic

What the pipeline produced

- A native KiCad 9 `.kicad_sch` — opened directly in Eeschema, not imported or converted
- **22 components, 68 nets, 126 pads**
- Footprints resolved from KiCad's own libraries, not guessed by the model
- Power rails drawn in ST's own convention, on ST's grid
- A matching `.net` file, which is what Phase 2 consumes

Verification status

KiCad ERC	✓ 0 errors, 0 warnings
Independent verifier	✓ clean
Floating pins	✓ all accounted for
Duplicate drivers	✓ none

The claim this supports

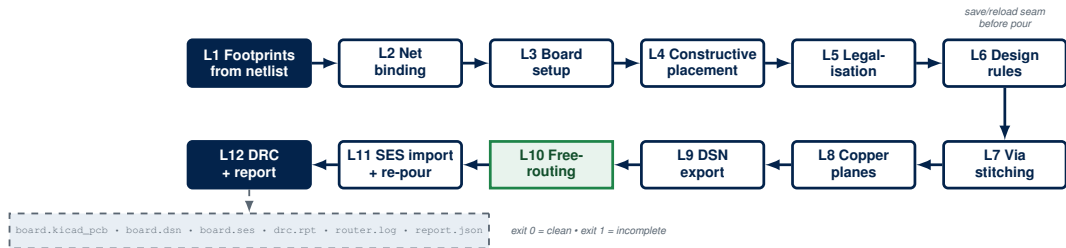
Not "an LLM drew a schematic." Rather: **an LLM proposed a netlist, and every downstream artifact was produced and checked by tools that can refuse it.**

Output 1 — what the generator decided, rail by rail

Rail	Pins	Caps	Decision	Why
VREF_PLUS	1	1	wired A-rail	stacks fit the free space
VBAT	1	1	wired A-rail	stacks fit the free space
VDDA	1	2	wired A-rail	stacks fit the free space
VDDIO2	1	1	wired A-rail	stacks fit the free space
VDDUSB	1	1	wired A-rail	stacks fit the free space
VDD_3V3	3	8	labelled B-rail	would cross other pins
VDD12	2	2	labelled B-rail	would cross other pins

This table *is* the routability rule executing. Nothing here was chosen by hand, and nothing was chosen by the model — the geometry decided it, and the decision is reproducible.

Phase 2 — layout and routing pipeline (12 stages)



One command, one timestamped directory. Netlist in, routed board out — no GUI, no manual step, machine-readable verdict.

Board & stackup (L3)

- 4-layer stackup; inner layers declared `LT_POWER` (a plane-layer declaration bug cost real DRC accuracy until found)
- DFM floors taken from the fab's published spec (PCB Power)

Placement (L4–L5)

- Constructive placement driven by **per-edge decap lane assignment**, using the *real ball coordinates* from the WLCSP-72 and LGA-14 footprints
- Force-directed legaliser: simultaneous displacement accumulation, stagnation escape by axis-flipping, epsilon convergence (*converged in 16 iterations*)

Design rules (L6)

- Tiered provenance for every constant: **fab floor** > **IPC-2221** > **policy**
- 4 netclasses, 68 nets assigned

Stitching & pours (L7–L8)

- Via stitching with combined copper *and* hole-clearance geometry
- Idempotent zone replacement for the copper pours

Routing bridge (L9–L11)

- Native `ExportSpecctraDSN / ImportSpecctraSES`
- Freerouting invoked headless as a subprocess

The derived router clearance — a worked example

Problem

The DSN format carries **copper-to-copper** clearance. It does **not** carry KiCad's **hole-to-copper** rule — so a router that satisfies the DSN can still produce a board that fails KiCad's DRC.

Solution — inflate the exported clearance so one rule implies the other

$$C_{\text{router}} = \max(C_{\text{copper}}, C_{\text{hole}} + r_{\text{hole}} - r_{\text{via}}) = \mathbf{0.1250 \text{ mm}}$$

Satisfying copper-to-copper at 0.1250 mm now *automatically* satisfies hole-to-copper. Applied **in memory** with immediate revert — netclass definitions live in `.kicad_pro` and do not survive a save/reload round-trip.

And the number was measured, not chosen: 0.09 mm → 6 unrouted, 14 hole violations • 0.1250 mm → 10 unrouted, 0 violations

The shape of most of the engineering here: not "call the API", but *find the semantic gap between two tools and close it with a derivation you can defend.*

Output 2 — the board that comes out

Property	Value
Board area	$\approx 625 \text{ mm}^2$
Layers	4 (inner = LT_POWER)
Assembly	single-sided
Copper pour, layer 4	GND
Copper pour, layer 6	/VDD_3V3
Stitching vias	40 placed, 1 unreachable
Traces after routing	94
Vias, total	44
Router clearance	0.1250 mm derived
Fab floor clearance	0.09 mm

What the placer actually did

Read the real ball coordinates out of the WLCSP-72 and LGA-14 footprints, assigned each decoupling capacitor to a lane on the edge of U1 nearest its own supply ball, then let the force-directed legaliser resolve the overlaps.

It converged in **16 iterations**.

What it did not do

! No bottom-side placement — the placer has no concept of a second side.

! No intentional trace widths — see Part 3.

Output 3 — the machine-readable verdict

```
>>> [L1/12] Footprints from netlist
      components: 22 | nets: 68 | pads: 126
>>> [L5/12] Legalization
      converged in 16 iterations
>>> [L6/12] Design rules
      classes: 4 | nets assigned: 68
>>> [L7/12] Via stitching
      stitched: 40 | unstitched: 1
      UNREACHABLE U1 pad H8 (/VDD_3V3)
>>> [L8/12] Copper planes
      poured GND on layer 4
      poured /VDD_3V3 on layer 6
>>> [L9/12] DSN export
      router clearance: 0.1250 mm
>>> [L10/12] Autorouting
      router ok: True | unrouted: 10
>>> [L11/12] SES import + re-pour
      traces: 94 | vias: 44
>>> [L12/12] DRC
      unconnected: 10 | copper: 0 | silk: 10
=== INCOMPLETE === exit code: 1
```

Honest failure is a feature

The run above is a **correct** result, not a regression.

10 unconnected items means the board is *not* clean — so the pipeline says *INCOMPLETE* and returns `exit 1`.

Why this matters

A flow that reports success optimistically is worse than no automation: it moves error discovery to the fab.

Every stage either proves its result or names what it could not do (UNREACHABLE U1 pad H8).

Where we stand — the scorecard

Metric	Result	Status
ERC errors (schematic)	0	✓
Independent verifier	clean	✓
Components / nets / pads	22 / 68 / 126	✓
Pads bound to nets	126 / 126	✓
Courtyard violations	0	✓
Copper clearance violations	0	✓
Via stitching	40 placed, 1 named unreachable	!
Unconnected items	10 (measured trade for 0 violations)	!
Silkscreen warnings	10 (deferred, cosmetic)	!
Manual/GUI steps required	0	✓

Phase 1 complete. **Phase 2 complete as a pipeline**, with a named, understood residual: the WLCSP-72 inner-ball fan-out.

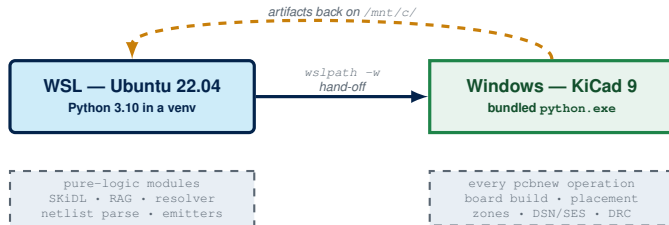
Part 2

Replicating the work

Environment, dependencies, SKiDL, KiCad, Freerouting, LLM APIs

15 minutes

The environment split — and why it exists



Why not one machine?

`pcbnew` is a SWIG binding into KiCad's C++ core. It must run under the *same* interpreter KiCad ships. Mixing it with a Linux venv gives ABI mismatches that fail in confusing ways.

Practical rule

Anything that is *pure logic* → WSL Python.
Anything that *touches a board object* → KiCad's Windows `python.exe`.

Step 1 — base environment

A. WSL + Python

```
wsl --install -d Ubuntu-22.04 # from Windows PowerShell
sudo apt update && sudo apt install -y python3.10 python3.10-venv python3-pip git

mkdir -p ~/LLM_Driven_Schematic_Gen && cd ~/LLM_Driven_Schematic_Gen
python3 -m venv venv
source venv/bin/activate
python -V # expect: Python 3.10.x
```

B. KiCad 9 on the Windows side

```
# install KiCad 9.0 from kicad.org (Windows installer)
# then verify the symbol libraries are visible from WSL:
ls "/mnt/c/Program Files/KiCad/9.0/share/kicad/symbols/" | head
"/mnt/c/Program Files/KiCad/9.0/bin/python.exe" -c "import pcbnew; print(pcbnew.Version())"
```

The install path contains a space — **always quote it**. Unquoted, it fails silently in half of all shell contexts.

Step 2 — Python dependencies

```
source ~/LLM_Driven_Schematic_Gen/venv/bin/activate

# --- netlist generation -----
pip install skidl==2.2.3 # Python -> netlist + ERC()
pip install anthropic==0.115.1 # primary LLM provider
pip install openai==2.46.0 # NVIDIA NIM (OpenAI-compatible)

# --- retrieval-augmented grounding -----
pip install pymupdf4llm==1.28.0 # layout-aware PDF -> markdown
pip install pymupdf-layout==1.28.0
pip install chromadb==1.5.9 # vector store
pip install sentence-transformers==5.6.0

pip freeze > requirements.txt # these are the tested pins
```

Do not skip `pymupdf4llm`

Plain PyMuPDF turns the STM32 ball-assignment tables into mojibake. The model is then grounded on garbage, and the failure surfaces three stages later as an invalid pin name.

SKiDL's role

SKiDL is the *intermediate representation*: the LLM writes Python; SKiDL executes it into a netlist and runs `ERC()`. The LLM never writes a file format.

Step 3 — Freerouting and Java

```
# Windows side: install a JRE, then confirm
java -version

# place the Freerouting jar somewhere stable, e.g.
# C:\tools\freerouting\freerouting.jar

# the pipeline invokes it headless as a subprocess:
java -jar freerouting.jar -de board.dsn -do board.ses
```

The bridge is pcbnew-native

KiCad 9 exposes both halves in Python —
`pcbnew.ExportSpecctraDSN` and
`pcbnew.ImportSpecctraSES`.

No GUI export step: board → DSN →
Freerouting → SES → board → DRC.

Verify before you trust

Check the JRE version your Freerouting build
requires — recent releases track new JREs
and fail with an opaque error on older ones.

Step 4 — LLM API configuration

Provider A — Anthropic (baseline)

```
export ANTHROPIC_API_KEY=sk-ant-... # from console.anthropic.com
# MODEL = "claude-sonnet-4-6"
# max_tokens = 4000
```

Provider B — NVIDIA NIM (OpenAI-compatible, free tier)

```
export NVIDIA_API_KEY=nvapi-... # from build.nvidia.com
# base_url = "https://integrate.api.nvidia.com/v1"
# model = "z-ai/glm-5.2"
# temperature low + fixed seed for reproducible codegen
```

One switch, not a migration

A single `PROVIDER` flag selects the branch. Both branches return `(text, in_tokens, out_tokens)`, so everything downstream — including the cost log — stays provider-neutral.

Two things people get wrong

- A **Claude Pro** subscription is not **API access** — the API bills separately.
- The sample `temperature=1` in most quickstarts is wrong for code generation.

Step 5 — run the flow

Build the RAG index (once per datasheet set)

```
source ~/LLM_Driven_Schematic_Gen/venv/bin/activate
cd ~/LLM_Driven_Schematic_Gen/rag
python build_rag_v2.py # pymupdf4llm extract -> chunk -> embed -> Chroma
```

Phase 1 — prompt to verified schematic (one command)

```
python pipeline.py --name spi_auto --output-root /mnt/c/.../pcbgen_outputs
python pipeline.py --name spi_auto --skip-generate # re-serialise, no tokens
```

Phase 2 — netlist to routed board (KiCad's Windows Python)

```
"/mnt/c/Program Files/KiCad/9.0/bin/python.exe" \
"$ (wslpath -w ~/LLM_Driven_Schematic_Gen/rag/layout_pipeline.py)" \
'C:\...\joined_e2e\joined_e2e.net' "$ (wslpath -w $RUN_DIR)"
echo "exit code: $?" # 0 = clean, 1 = incomplete
```

Phase 1 runs entirely in the venv. Phase 2 must be launched with KiCad's own interpreter — never the venv Python.

Verified environment — the tested combination

Component	Version
KiCad / pcbnew	9.0.8
OS	Ubuntu 22.04 (WSL)
Python	3.10
skidl	2.2.3
anthropic	0.115.1
openai	2.46.0
pymupdf	1.28.0
pymupdf-layout	1.28.0
pymupdf4llm	1.28.0
chromadb	1.5.9
sentence-transformers	5.6.0

Why pin all of it

`pcbnew` is a SWIG binding to a specific KiCad build. A minor KiCad bump can change enum names, default clearances, or whether a call exists at all — and the failure shows up as a wrong *number*, not an exception.

Still unpinned — to close before publication

- ! Freerouting jar version
- ! JRE version on the Windows side
- ! Fab DFM spec revision (PCB Power)

Each of these silently changes results. They belong in `report.json`, recorded per run.

Module map — the real repository

Layer	Modules	Responsibility
Grounding	<code>build_rag_v2</code> <code>query_rag</code> <code>kicad_symbol_parser</code> <code>symbol_resolver</code>	Retrieval; parse <code>.kicad_sym</code> ; authoritative pin table
Generation	<code>generate_spi_v2</code> <code>generate_st_circuit</code> <code>generate_i2c_circuit</code> <code>lsm6dsm_stm32l476_spi_v2</code> <i>(generated)</i> • <code>...CLAUDE_BASELINE</code> <i>(frozen known-good)</i>	Prompt assembly, provider switch, SKiDL codegen, ERC gate
Schematic	<code>netlist_parser</code> <code>netlist_reader</code> <code>component_model</code> <code>geometry</code> <code>grouping_roles_placement</code> <code>power_wiring</code> <code>schematic_emitter</code> <code>kicad_sch_builder</code> <code>kicad_sch_writer</code> <code>pipeline</code>	Netlist → geometry → ST-style power wiring → <code>.kicad_sch</code>
Verification	<code>verify_schematic</code> • 6 × <code>check_*</code>	Union-find on drawn connectivity; clearance audits
Layout	<code>board_setup</code> <code>board_builder</code> <code>placement_engine</code> <code>legalize</code> <code>design_rules</code> <code>rules_apply</code> <code>via_stitch</code> <code>via_geom</code> <code>zones</code> <code>export_dsn</code> <code>routing</code> <code>analyze_ses</code> <code>layout_pipeline</code> <code>run_context</code>	Placement, rules, stitching, pours, DSN/SES bridge, DRC + report

Plus harnesses: `layout_stage2..12_*.py` per-stage runners • 13 `probe_*.py` • 7 `test_*.py`

The 13 probe scripts — discipline made visible

Each `probe_*.py` is a question asked of the tooling **before** any code was written against it:

```
layertype  
netclass_api  
netclass_assign  
roundtrip_nc  
zone • pour_inmem  
inmem_courtyard  
via • via_in_pad  
ses_import  
java_subprocess  
clearance_cost  
cachefix
```

```
can I declare a plane layer?  
does the API expose netclasses?  
does assignment stick?  
does it survive save/reload?  
can I pour in memory?  
is courtyard data live?  
what via geometry is legal?  
does import preserve prior work?  
can I drive Freerouting headless?  
what does clearance cost me?  
is the symbol cache stale?
```

The rule behind them

Never write final code against an assumption about a library API, tool output, or environment state. Probe first; build on the confirmed result.

What it bought

`roundtrip_netclass` is why we know netclasses live in the `.kicad_pro` file.

`pour_inmem` is why the save/reload seam exists instead of a mysterious C++ crash.

Each probe is a bug that never happened.

Gotchas that cost real time — read before you build

Trap	What actually happens / the fix
Stale .kicad_pro	KiCad 9 stores <i>netclass definitions in the project file</i> , not the board. SaveBoard will not overwrite an existing .kicad_pro, so a stale file silently overrides your computed design rules and invalidates every measurement. Delete it before saving.
Netclass round-trip	Netclasses do not survive save → reload through a temp path. Do the DSN clearance inflation in memory , with an immediate revert.
ZONE_FILLER crash	Hard-crashes at the C++ level on an in-memory board, with <i>no Python traceback</i> . BuildConnectivity() does not help. Insert a deliberate save/reload seam before pouring .
Plane layer type	Inner layers must be declared LT_POWER, or the DRC engine reports misleading clearance results.
Path with a space	C:\Program Files\KiCad\9.0 — quote it everywhere, and use wslpath -w for every hand-off.
Unsourced constants	A POWER clearance of 0.15 mm was invented, not sourced. Removing it and grounding to the fab floor (0.09 mm) recovered four connections . <i>No unsourced constants in a release.</i>

What is already deterministic

- Every run writes to a **timestamped directory** with all six artifacts.
- `report.json` records stage-by-stage results, not just the final verdict.
- Exit code carries the verdict: 0 clean, 1 incomplete.
- The legaliser converges to an epsilon, not a fixed iteration count — same input, same output.

What still varies

- ! The LLM stage. Anthropic's API exposes no seed; NVIDIA NIM does — so GLM runs are *more* reproducible than the Claude baseline.
- ! Freerouting is heuristic; different runs may route differently, though the DRC verdict has been stable.

Publication checklist

Pin `requirements.txt`; record KiCad build, Freerouting version, JRE version, model ID and temperature/seed *in* `report.json`.

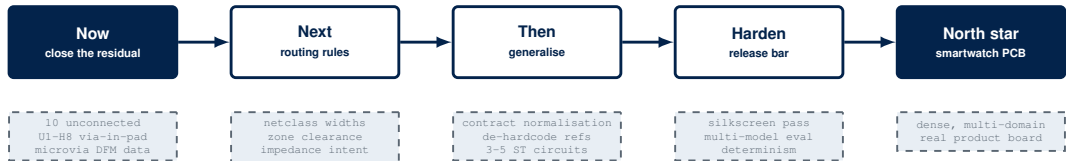
Part 3

Next tasks

Open gaps, the release bar, and the north star

15 minutes

Roadmap



Task 1 — close the 10 unconnected

Root cause

All ten are **WLCSP-72 inner-ball escape routes**. At 0.4 mm ball pitch, the keep-out ring leaves no legal channel for a through-via to reach U1's inner balls.

We already know the price of the alternative

Router clearance	Unrouted	Hole violations
0.09 mm	6	14
0.1250 mm	10	0

The plan

- **Via-in-pad** for U1–H8 — geometry proven: a 0.225 mm pad permits a via up to 0.395 mm.
- Choose the via strategy *explicitly*: microvia / blind-buried vs. through.
- **! Blocked**: awaiting the fab's microvia drill, annular ring and hole-clearance specs.

The general principle

Fine-pitch BGA and WLCSP escape is a **stackup decision, not a router decision**. No autorouter can solve a fan-out that the layer count forbids — the options are more layers, microvias, or unused balls.

Why this is the right first task

The 0.09 mm board connects four more nets and is **not manufacturable**. We took the manufacturable one.

Not a bug — but it is the only thing between this run and `exit 0`, and closing it means changing the *stackup*.

Task 2 — routing rules: the honest gap

Self-assessment

We set `m_TrackMinWidth = 0.09mm` — but that is a **minimum the DRC enforces**, not the width the **router uses**. We never created netclasses with actual widths, so Freerouting routed at the DSN default.

Every trace on the current board is a width nobody specified. Zone clearance had the same problem — it sat at KiCad's unsourced 0.5 mm default until it was grounded to the fab floor. Trace widths are the last one left.

What to build

- Real netclasses with *intentional* widths per class: power / ground / signal / high-speed.
- Current-carrying width from IPC-2221 for the supply rails.
- Explicit zone local clearance — no inherited defaults.

What comes after

- Length matching and impedance intent for the SPI bus (not needed at these speeds — needed for the north star).
- Differential-pair support.
- Per-class via rules.

Task 3 — generalisation

Generation-contract normalisation

The multi-model experiment proved the pipeline is brittle to naming variants:

VSS vs GND • VDD vs VDD_3V3 • VREF+ vs VREF_PLUS • 100n vs 100nF

Add a normalisation layer so layout never breaks on off-contract input.

De-hardcode the placer

Circuit-specific reference designators (the R3/R4-style hardcodes) must go. The layout stage has to be **topology-agnostic** to be a product.

Broaden the benchmark

Prove the full flow on **3–5 diverse ST circuits**, not one board. First addition: an STM32F0 motor-controller topology — different package, different rail structure, different constraints.

Double-sided placement

Today every component is top-side; the placer has **no concept of a bottom side**. Real designers push passives to the solder side when the top crowds. Needs per-side collision sets and `B.CrtYd` courtyards.

Task 4 — multi-model evaluation

A controlled comparison: identical prompt, identical RAG context, different model.

Observation	Detail
Format contract held	GLM-5.2 produced valid SKiDL Python, no markdown leakage ✓
ERC-invisible pin error	A signal pin was mis-named: syntactically valid, electrically wrong. ERC did not catch it. ✗
Off-contract net names	VSS/VDD/VREF+ instead of the contract — broke the power-wiring stage ✗
Missing components	Absent resistors tripped the placement hardcodes ✗

Conclusion

Multi-model validation is not optional — a single model's idiosyncrasies get silently encoded into the pipeline's assumptions. This also gives the first dataset for a paper: *which classes of hardware error survive ERC?*

Definition of done — the release bar

Layout stage is release-ready when *all* of these hold, across the benchmark set

- One command: any Phase-1 schematic → routed board
- **0 unconnected**
- DRC-clean (copper, courtyard, silkscreen)
- No unsourced constants anywhere
- No GUI steps
- No hardcoded reference designators
- A clear machine-readable pass/fail report
- Deterministic, with honest failure behaviour

Acceptance gate, unchanged since Phase 1: every board must pass *two independent judges* —
KiCad's own DRC **and** our independent verifier. Neither alone is trusted.

The target: a smartwatch PCB Chosen

deliberately, because it breaks every simplifying assumption the current pipeline makes:

- Dense, irregular board outline — not a rectangle
- Multi-domain power: charger, PMIC, multiple rails
- Mixed signal: RF, analogue sensors, digital bus
- Genuine two-sided assembly
- Fine-pitch escape everywhere, not just at U1

Asks from ST

- Footprint mappings for the target part families
- Recommended stackups
- Placement guidelines / checklists
- `.kicad_dru` rule files, if they exist
- 2–3 reference boards as ground truth

Open question for discussion

Where should the LLM's authority end? Today it proposes only the *netlist*. Should it also propose *placement intent* — or does that break the verifiability that makes this flow trustworthy?

What exists today

- A working, headless, prompt-to-board pipeline for an ST benchmark circuit
- 0 ERC errors; 126/126 pads bound; 0 copper and courtyard violations
- Two-judge verification catching faults ERC cannot see
- Machine-readable reporting with honest failure

What is left

- 10 unconnected — a stackup decision, blocked on fab data
- Intentional routing rules (widths, clearances)
- Generalisation across circuits and models
- Silkscreen, double-sided placement, determinism

The thesis this project tests: an LLM is safe at the front of a hardware flow precisely when every stage behind it is owned by an authoritative tool that can refuse its output.

Questions

15 minutes

Dushyant Singh • Semiconductor Chip Design Club, SRM IST

Backup slides follow

Quantity	Value
Components / nets / pads	22 / 68 / 126
Legaliser convergence	16 iterations
Netclasses / nets assigned	4 / 68
Via stitching	40 stitched, 1 unstitched (U1 H8, /VDD_3V3)
Copper pours	GND on layer 4, /VDD_3V3 on layer 6
Derived router clearance	0.1250 mm
Fab floor copper clearance	0.09 mm
Router result	ok, 10 unrouted
After SES import	94 traces, 44 vias
DRC	10 unconnected, 0 copper violations, 10 silk warnings
Verdict / exit code	INCOMPLETE / 1

How `verify_schematic.py` works

1. Parse the emitted `.kicad_sch`: wire segments, junctions, labels, symbol pin coordinates.
2. Build a **union-find** structure keyed on geometric coincidence — wire endpoints, label anchors, pin positions.
3. Collapse into connected components: this is the *drawn* connectivity.
4. Diff drawn connectivity against the *netlist* spec.

Fault classes it detects

- Rail merges (two supplies unified by a stray wire)
- Signal-on-power shorts
- Dangling labels
- Missing connections

Why geometry, not the netlist

KiCad's ERC reads the schematic's *intent*. The verifier reads its *geometry*. They fail differently — which is the entire point of having both.